

machine learning applications. Lisp, Haskell, and Scala are significant programming languages in the domain of AI because of the programming paradigm they support. Functional programming features are extremely beneficial when developing AI and machine learning applications. Although Python does support certain functional programming features, such as lambda functions, it is not enough, which underscores the critical importance of programming languages like Lisp, Haskell, and Scala.

Programming languages like C++ and Java are considered for developing AI when requirements like speed, low-level control, and scalability come into the picture. Since C++ and Java are quite well-known, let us move on to discussing other languages.

R is a programming language for statistical computing and is preferred over Python when strong statistical computing capabilities are required. For more details, I urge you to go through the ongoing series on R currently featured in *Open Source For You*.

In my opinion, a comprehensive discussion on the development of AI-based software should include references to proprietary programming languages and tools like MATLAB, Wolfram Mathematica, etc. However, I am aware that many advocates of free and open source software consider such tools as unacceptable. Personally, I align with the moderate faction of the free and open source software community, and do not entirely discourage the use of proprietary software. However, to avoid any controversy, I will talk about Scilab and GNU Octave, two programming languages that have the ability to mimic MATLAB effectively.

Before we proceed any further, I would like to introduce a textbook titled 'Neural Data Science: A Primer with MATLAB and Python' by Nylen and Wallisch. This textbook provides examples in both Python and MATLAB, which makes it easier to switch between

the two programming languages. Furthermore, I have tested the code in this textbook using both Scilab and GNU Octave interpreters to ensure their conformity with MATLAB. These comparisons revealed that Scilab has lower syntactic compatibility with MATLAB than GNU Octave.

Now, let us discuss Scilab. It is a numerically-oriented programming language and a free and open source numerical computational package. Scilab is an excellent tool for developing AI and machine learning-based applications due to its ease of learning and rich set of libraries. To install Scilab on Ubuntu, run the command '*sudo apt-get install scilab*' in your terminal, and execute the

```
-->A = [1 1 1]
A =

    1.    1.    1.

-->B = [11; 22; 33]
B =

    11.
    22.
    33.

-->A*B
ans =

    66.

-->B*A
ans =

    11.    11.    11.
    22.    22.    22.
    33.    33.    33.
```

Figure 2: Matrix multiplication using Scilab

```
octave:1> A = [1 2; 3 4; 5 6]
A =

     1     2
     3     4
     5     6

octave:2> transpose(A)
ans =

     1     3     5
     2     4     6
```

Figure 3: Transpose of a matrix using GNU Octave

command '*scilab*' to run the Scilab interpreter. Figure 2 shows a simple example of multiplying two matrices using Scilab. Note that matrix A is of order 1x3 (1 row and 3 columns) and matrix B is of order 3x1 (3 rows and 1 column). Therefore, the product matrix AB has an order of 1x1, with the single element being 66, while the product matrix BA has an order of 3x3. This is a valid MATLAB code as well, but even basic tasks expose the syntactic incompatibility of Scilab with MATLAB. For instance, the perfectly valid MATLAB code '*C = transpose(B)*' produces the following error in Scilab: '*Undefined variable: transpose*'. As a result, potential users of Scilab should exercise extreme caution when using MATLAB code.

Let us now discuss GNU Octave, a free and open source programming language primarily used for scientific computing and numerical computation. Its compatibility with MATLAB, as well as its comprehensive set of tools, make it a powerful programming language for developing AI and machine learning-based applications. To install GNU Octave on Ubuntu, simply run the command '*sudo snap install octave*' in your terminal, and to run the GNU Octave interpreter, execute the command '*octave*'. In Figure 3, a simple example of finding the transpose of a matrix using GNU Octave is shown. Note that matrix A has order 3x2 (3 rows and 2 columns) and therefore, the transpose of matrix A has order 2x3 (2 rows and 3 columns). It is worth noting that, unlike Scilab, this code (which is compatible with MATLAB) works perfectly fine with GNU Octave.

In fact, MATLAB-compatible code works almost flawlessly with GNU Octave, unlike Scilab. The syntactical differences between MATLAB and GNU Octave are relatively rare, but there are a few that might cause trouble. Figure 4 shows some minor syntactic differences